SHELID AI

SMS SPAM DETECTION SYSTEM

FINAL PROJECT REPORT

A machine-learning web application for classifying SMS messages as spam or not spam.

Prepared by: M S Suhan, Gaurav Vyas, Shubam Parmar, Kunal Roy
Institution: Indian Institute of Information Technology Vadodara
Date: 28-10-2026

Abstract

Shelid AI is a web-based Web spam detection application developed with Python and Streamlit. It accepts a text message from a user, preprocesses the message, converts the cleaned text into numerical features using a previously fitted TF-IDF vectorizer, and passes those features to a trained machine-learning classifier. The application displays a result indicating whether the message was classified as spam or not spam. Natural Language Toolkit (NLTK) utilities support tokenization, English stopword removal, and Porter stemming. The project demonstrates an end-to-end workflow connecting text preprocessing, a saved machine-learning model, and an interactive web interface. Actual predictive performance depends on the training data and must be established through evaluation; no accuracy figure is claimed in this report.

Keywords: SMS spam detection, text classification, TF-IDF, Multinomial Naive Bayes, NLTK, Streamlit.

1. Introduction

Web messages are widely used for personal communication, notifications, and service updates. However, unwanted messages can include unsolicited advertising, misleading offers, and phishing attempts. A spam classifier can help users flag messages that resemble patterns learned from labelled examples. Shelid AI provides a simple interface for exploring this task with traditional machine-learning methods.

1.1 Problem Statement

The project aims to build an accessible application that takes an SMS message as input and predicts whether it belongs to the spam or non-spam class using a trained text-classification model.

1.2 Objectives

- Build a usable interface for entering and testing SMS messages.
- Apply consistent text preprocessing before feature extraction.
- Use a fitted TF-IDF vectorizer to represent text as numerical features.
- Use a trained classifier to generate a binary prediction.
- Present results clearly and communicate that predictions may be imperfect.

2. Scope and Requirements

The current scope covers single-message classification in a local or hosted Streamlit application. It does not automatically block SMS messages on a mobile device, verify a sender's identity, or guarantee that a message is safe.

2.1 Functional Requirements

- Accept a text message from the user.
- Allow sample messages to be loaded for demonstration.
- Preprocess text and transform it with the saved vectorizer.
- Return and display the classifier's predicted class.
- Handle empty input and report initialization or prediction errors.

2.2 Non-functional Requirements

- Simple, responsive user interface.
- Reasonable response time for a single message.
- Maintainable separation between preprocessing, model loading, and UI logic.

- Keep model and vectorizer artifacts compatible and load them only from trusted sources.

3. Technology Stack

Technology	Purpose
Python	Core application and model-inference logic.
Streamlit	Interactive web application and user interface.
scikit-learn	TF-IDF feature extraction and machine-learning classification.
NLTK	Tokenization, English stopwords, and Porter stemming.
Pickle	Serialization and loading of the fitted model and vectorizer.
HTML/CSS	Custom styling for the dark, modern interface.

4. System Architecture

The application follows a sequential inference pipeline. The trained model and vectorizer are created during the training workflow and saved as separate files. At application startup, Streamlit loads these artifacts. For each submitted message, the app performs the same intended preprocessing used during training, applies the vectorizer, and obtains a class prediction from the classifier.

Stage	Description
1. User input	The user enters an SMS message or selects a sample.
2. Preprocessing	Lowercase, tokenize, remove English stopwords and non-alphanumeric tokens, then stem.
3. TF-IDF	Transform the processed text with the fitted vectorizer.
4. Classification	Pass the feature vector to the trained model.
5. Output	Display Spam or Not Spam based on the predicted label.

4.1 Main Components

Component	Responsibility
app.py	Defines the Streamlit interface and coordinates inference.
vectorizer.pkl	Stores the fitted TF-IDF vectorizer and vocabulary.
model.pkl	Stores the fitted classification model.
Preprocessing function	Converts raw text into the representation expected by the model.

5. Methodology

5.1 Text Preprocessing

The inference code converts the message to lowercase, tokenizes it using NLTK, keeps alphanumeric tokens that are not English stopwords, and stems the remaining tokens with PorterStemmer. The resulting tokens are joined into a processed string. The training pipeline should apply the same transformations to its data.

5.2 TF-IDF Feature Extraction

Term Frequency–Inverse Document Frequency (TF-IDF) maps text into numerical features based on term frequency and how informative a term is across the training corpus. The saved, fitted vectorizer is reused at inference time; fitting a new vectorizer on an individual message would produce incompatible features.

5.3 Classification

The application calls the loaded classifier's predict method on the TF-IDF feature vector. The deployed model is expected to use label 1 for spam and label 0 for not spam. This mapping must match the labels used during training. The notebook workflow considered Multinomial Naive Bayes and other classifier combinations; the exact model deployed is determined by the contents of model.pkl.

5.4 Result Presentation

The interface displays a warning for a spam prediction and a positive-style message when the model does not flag the message as spam. A not-spam prediction is not treated as proof that a message is legitimate.

6. Implementation and Deployment

6.1 Project Files

The runtime directory should contain app.py, model.pkl, and vectorizer.pkl. A requirements.txt file is recommended to make dependency installation reproducible. The model and vectorizer must be generated from a compatible training run.

6.2 Setup

```
Create a virtual environment: python -m venv venv
Activate on Linux/macOS: source venv/bin/activate
Install packages: pip install streamlit scikit-learn nltk
Start the app: streamlit run app.py
```

Streamlit prints a local address, commonly http://localhost:8501. In deployment, the hosting environment must include the required Python packages and both serialized artifacts. NLTK tokenizer data may also need to be available, depending on the NLTK version and preprocessing calls.

7. Testing and Validation

The following cases are recommended for functional testing. Fill in the actual outcomes after running the application with the final model. A successful UI run alone does not establish model accuracy.

Test case	Expected behaviour	Actual result
Empty input	Show a prompt and do not run prediction.	No result's
Typical spam example	Display the model's predicted class.	Spam
Ordinary personal message	Display the model's predicted class.	Not Spam
Long message	Handle within the configured input limit.	Trim Text and Detect
Missing model/vectorizer	Show a clear initialization error.	Cannot Predict Message
Unfitted model artifact	Fail visibly rather than silently predicting.	Untrained model.pkl

7.1 Model Evaluation

For a complete evaluation, use a held-out test set that was not used to train the model. Report accuracy, precision, recall, F1-score, and a confusion matrix. For spam detection, precision indicates how often messages predicted as spam are actually spam, while recall measures how many actual spam messages are detected. No numerical results are included because verified evaluation outputs were not provided.

8. Limitations and Security

- Prediction quality depends on the training dataset, label quality, class balance, and similarity between training and real-world messages.
- The current preprocessing may remove useful signals such as punctuation, URLs, or certain tokens.
- Messages in languages other than English may not be handled reliably.
- A not-spam prediction does not establish sender authenticity or guarantee safety.
- Pickle files can execute code when loaded. Load only artifacts from trusted sources.
- For public deployment, review logging, privacy, rate limits, dependency versions, and sensitive content users may submit.

9. Future Enhancements

- Publish measured test-set metrics and compare candidate models using the same split.
- Add prediction probabilities only when the chosen estimator supports meaningful probability estimates.
- Improve preprocessing through error analysis and validation against the training pipeline.
- Add automated unit tests for text transformation, model loading, and label mapping.
- Add optional prediction history with explicit privacy controls.
- Pin dependency versions and automate tests in continuous integration.
- Deploy to a hosting service with documented configuration and artifact management.

10. Conclusion

Shelid AI demonstrates an end-to-end text-classification application, from user-entered SMS content through preprocessing and TF-IDF feature extraction to a machine-learning prediction displayed in a web interface. The project combines Python, NLTK, scikit-learn, and Streamlit in a practical software deliverable. The next important step for a production-quality version is to validate the exact deployed artifact, ensure preprocessing matches training, and report measured performance on held-out data. The result should be treated as a helpful signal rather than a definitive security verdict.

References

- Streamlit documentation — https://docs.streamlit.io/
- scikit-learn documentation — https://scikit-learn.org/stable/
- NLTK documentation — https://www.nltk.org/
- Manning, Raghavan, and Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008 (TF-IDF background).
- Dataset - https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset